4.4 Long short-term memory (LSTM)

- the challenge to address long-term information preservation and short-term input skipping in latent variable models has existed for a long time
- one of the earliest approaches to address this was the *long short-term memory* (*LSTM*)
- it shares many of the properties of the GRU
- interestingly, LSTMs have a slightly more complex design than GRUs, but predate GRUs by almost two decades

4.4 Long short-term memory (LSTM)

- arguably LSTM's design is inspired by logic gates of a computer
- LSTM introduces a *memory cell* (or *cell* for short) that has the same shape as the hidden state (some papers consider the memory cell as a special type of hidden state), engineered to record additional information
- to control the memory cell, we need a number of gates
- one gate is needed to read out the entries from the cell; we will refer to this as the *output gate*

4.4 Long short-term memory (LSTM)

- a second gate is needed to decide when to read data into the cell; we refer to this as the *input gate*
- lastly, we need a mechanism to reset the content of the cell, governed by a *forget gate*
- the motivation for such a design is the same as that of GRUs, namely to be able to decide when to remember and when to ignore inputs in the hidden state, via a dedicated mechanism
- let us see how this works in practice

4.4 Long short-term memory (LSTM)

- just like in GRUs, the data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step, as illustrated in Figure 8
- they are processed by three fully-connected layers with a sigmoid activation function to compute the values of the input, forget, and output gates
- as a result, values of the three gates are in the range of $(0, 1)$

4.4 Long short-term memory (LSTM)

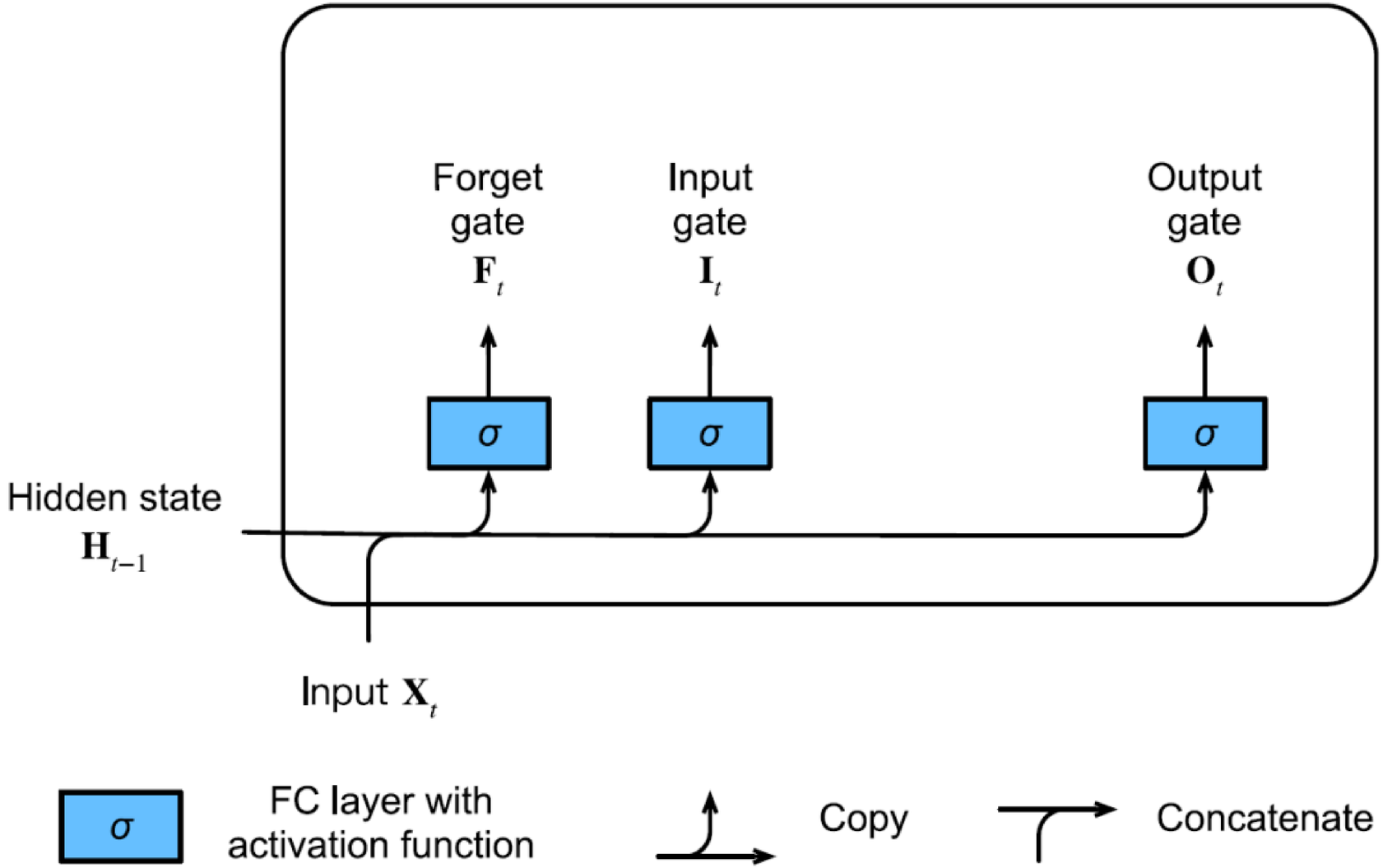


Figure 8: Computing the input gate, the forget gate, and the output gate in an LSTM model.

4.4 Long short-term memory (LSTM)

- mathematically, suppose that there are h hidden units, the batch size is n, and the number of inputs is d
- thus, the input is $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ and the hidden state of the previous time step is $\boldsymbol{H}_{t-1} \in \mathbb{R}^{n \times h}$
- correspondingly, the gates at time step t are defined as follows: the input gate is $\boldsymbol{I}_t \in \mathbb{R}^{n \times h}$, the forget gate is $\boldsymbol{F}_t \in \mathbb{R}^{n \times h}$, and the output gate is $\boldsymbol{O}_t \in \mathbb{R}^{n \times h}$
- they are calculated as follows:

$$
\begin{aligned}
\boldsymbol{I}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xi} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hi} + \boldsymbol{b}_i), \\
\boldsymbol{F}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xf} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hf} + \boldsymbol{b}_f), \\
\boldsymbol{O}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xo} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{ho} + \boldsymbol{b}_o),
\end{aligned}
$$

where $\boldsymbol{W}_{xi}, \boldsymbol{W}_{xf}, \boldsymbol{W}_{xo} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hi}, \boldsymbol{W}_{hf}, \boldsymbol{W}_{ho} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_i$, $\boldsymbol{b}_f, \boldsymbol{b}_o \in \mathbb{R}^{1 \times h}$ are bias parameters

4.4 Long short-term memory (LSTM)

- next, we design the memory cell
- since we have not specified the action of the various gates yet, we first introduce the *candidate* memory cell $\tilde{\boldsymbol{C}}_t \in \mathbb{R}^{n \times h}$
- its computation is similar to that of the three gates described above, but using a tanh function, with a value range of $(-1, 1)$ as the activation function
- this leads to the following equation at time step t:

$$\tilde{\boldsymbol{C}}_t = \tanh(\boldsymbol{X}_t \boldsymbol{W}_{xc} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hc} + \boldsymbol{b}_c),$$

  where $\boldsymbol{W}_{xc} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hc} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_c \in \mathbb{R}^{1 \times h}$ is a bias parameter
- a quick illustration of the candidate memory cell is shown in Figure 9

4.4 Long short-term memory (LSTM)

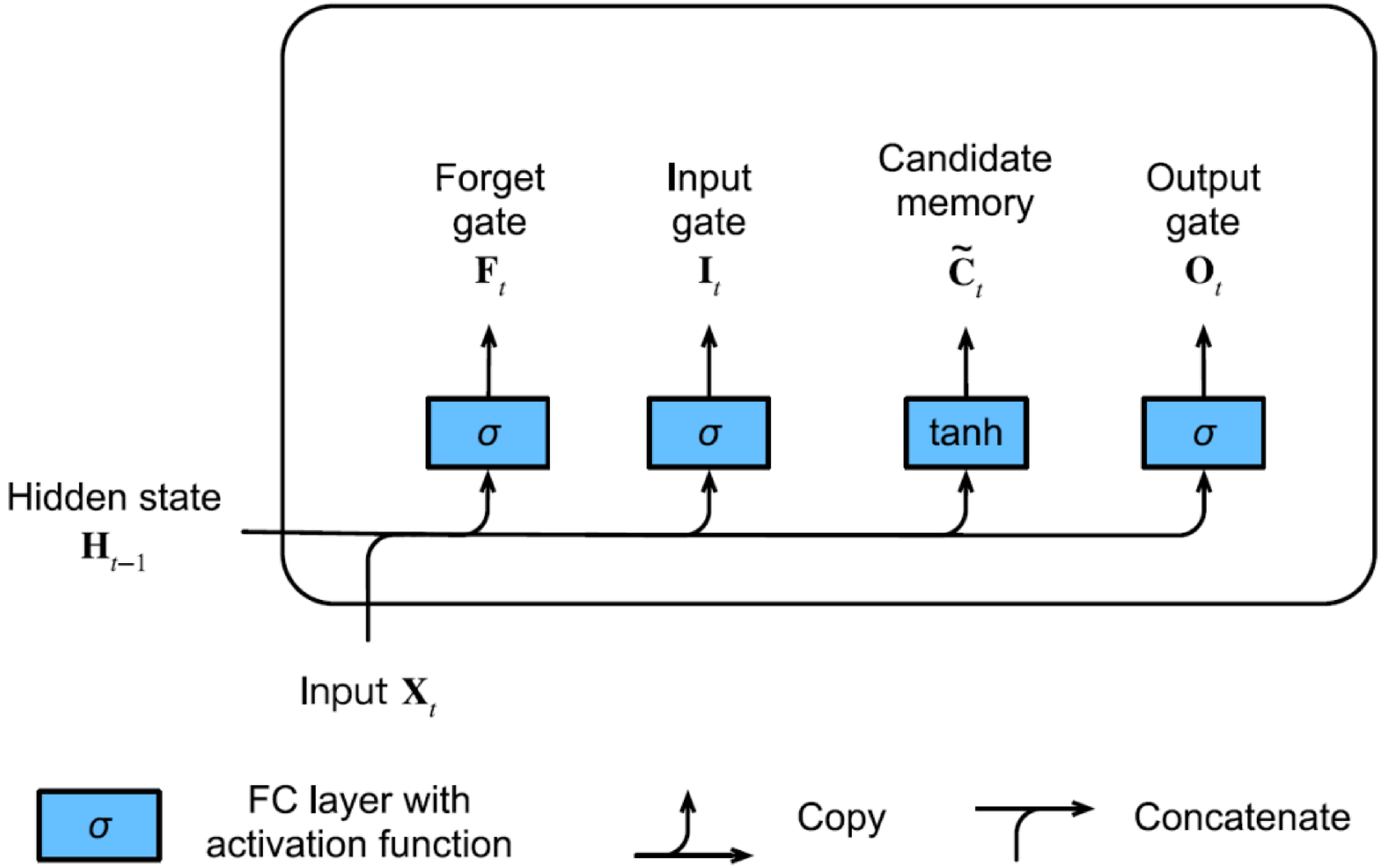


Figure 9: Computing the candidate memory cell in an LSTM model.

4.4 Long short-term memory (LSTM)

- in GRUs, we have a mechanism to govern input and forgetting (or skipping)
- similarly, in LSTMs we have two dedicated gates for such purposes: the input gate $\boldsymbol{I}_t$ governs how much we take new data into account via $\tilde{\boldsymbol{C}}_t$, and the forget gate $\boldsymbol{F}_t$ addresses how much of the old memory cell content $\boldsymbol{C}_{t-1} \in \mathbb{R}^{n \times h}$ we retain
- using the same element-wise multiplication trick as before, we arrive at the following update equation:

$$\boldsymbol{C}_t = \boldsymbol{F}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{I}_t \odot \tilde{\boldsymbol{C}}_t.$$

- if the forget gate is always approximately 1, and the input gate is always approximately 0, the past memory cells $\boldsymbol{C}_{t-1}$ will be saved over time and passed to the current time step
- this design is introduced to alleviate the vanishing gradient problem, and to better capture long range dependencies within sequences
- we thus arrive at the flow diagram in Figure 10

4.4 Long short-term memory (LSTM)

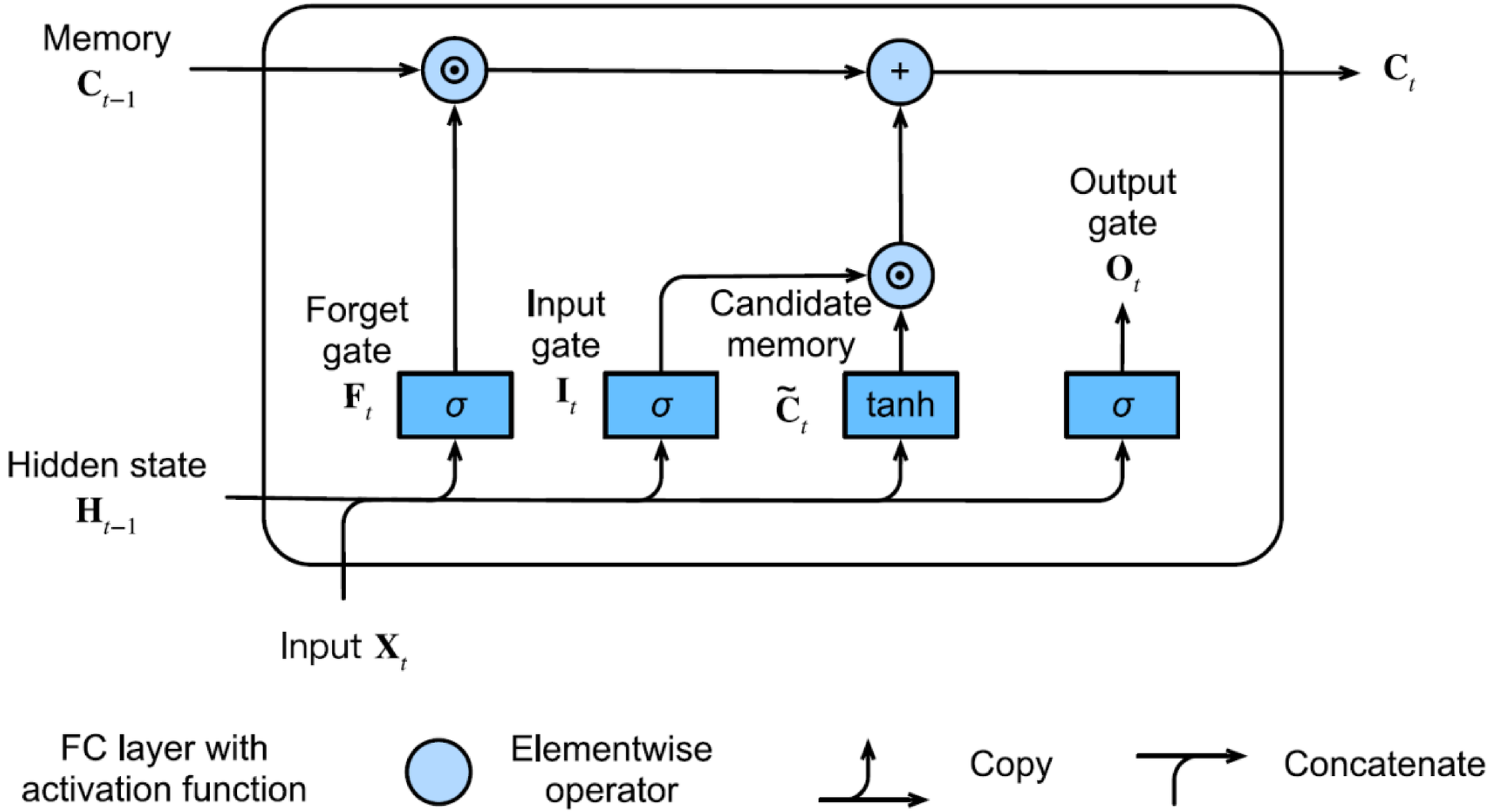


Figure 10: Computing the memory cell in an LSTM model.

4.4 Long short-term memory (LSTM)

- lastly, we need to define how to compute the hidden state $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$; this is where the output gate comes into play
- in LSTM, it is simply a gated version of the tanh of the memory cell
- this ensures that the values of $\boldsymbol{H}_t$ are always in the interval $(-1, 1)$:

$$\boldsymbol{H}_t = \boldsymbol{O}_t \odot \tanh(\boldsymbol{C}_t).$$

- whenever the output gate approximates 1, we effectively pass all memory information through to the predictor, whereas for the output gate close to 0, we retain all the information only within the memory cell, and perform no further processing
- Figure 11 gives a graphical illustration of the data flow

4.4 Long short-term memory (LSTM)

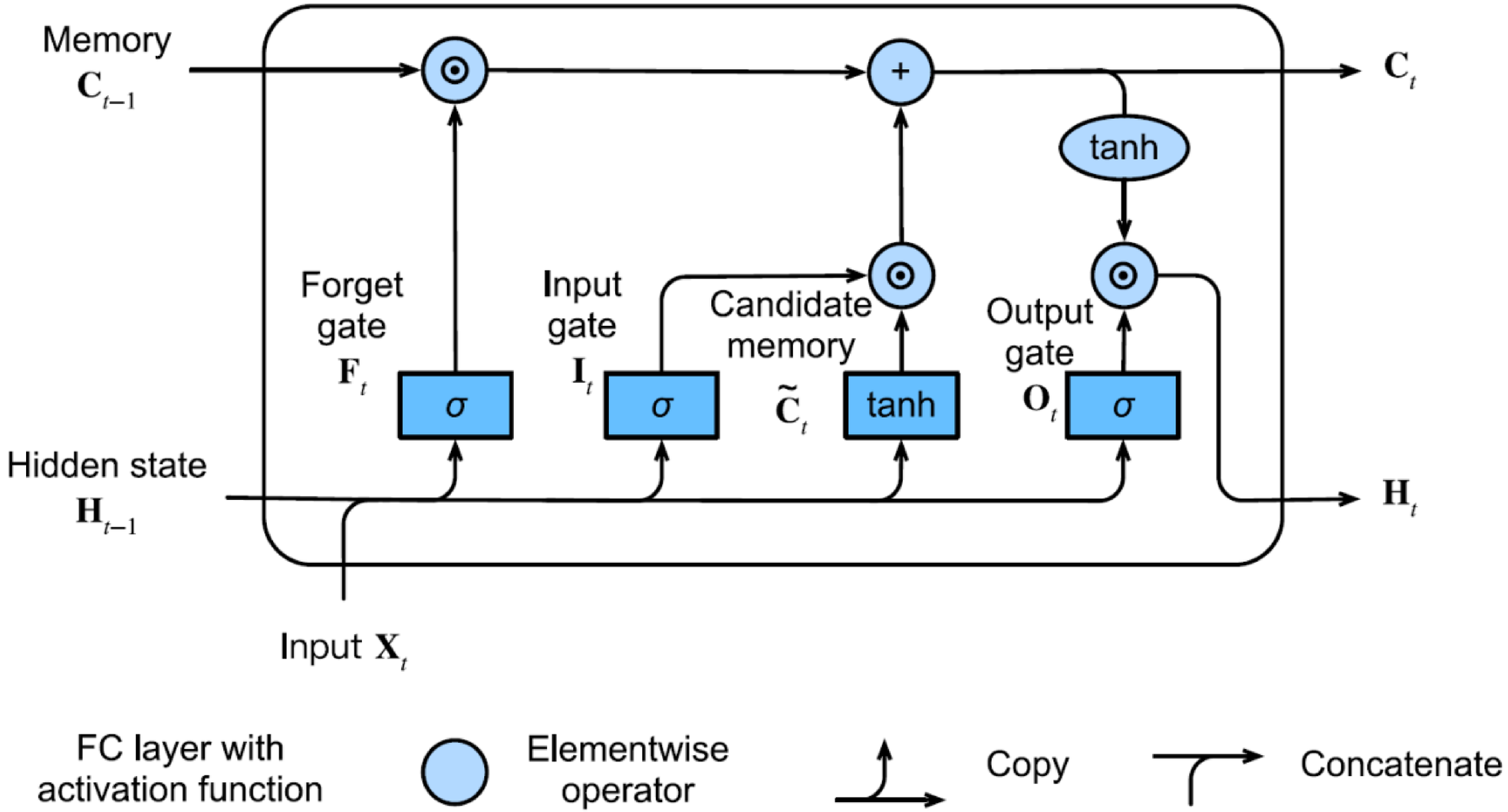


Figure 11: Computing the hidden state in an LSTM model.